MessageFormat v2 — Research

- Author: Alexey Lyakhov

Target: an `icu_messageformat` crate for the ICU4X project.
Branch: `feature/messageformat-v2`.
Date compiled: 2026-04-18.

This document gathers the background research that underlies the architecture and implementation plans (see `messageformat-v2-architecture.md` and `messageformat-v2-implementation-details.md`). It is the "why" — the other two are the "what" and the "how".

1. Specification status

- **Official name**: Unicode MessageFormat (MF2). During development the spec was commonly called "MessageFormat 2.0"; the Final Candidate text adopted the neutral name.
- **Normative source**: LDML 46.1 — `tr35-messageFormat.html`, part of CLDR Technical Report #35.
- **Status**: *Final Candidate*. Syntax and data model are stable; the `:date` / `:time` / `:datetime` functions are still marked *Draft*, `u:locale` is still *Draft*. Stable parts are recommended for implementation per the CLDR-TC.
- **Working group mirror** (editor's copy, test suite, exploration): `unicode-org/message-format-wg` (cloned locally at `/Users/alexeylyakhov/projects/message-format-wg`).
- **Related standardization**: the TC39 ECMA-402 `Intl.MessageFormat` proposal (stage 1 at time of writing) shadows MF2 semantics.

Existing implementations

Language	Project	Status	Notes
Java	`com.ibm.icu.message2` (ICU 76)	Tech preview	Reference formatter
C/C++	`icu::message2::MessageFormatter` (ICU 76)	Tech preview	Reference formatter
JavaScript / TS	`messageformat` v4+	Stable MF2-only release	Also polyfills the proposed `Intl.MessageFormat`
i18next	`i18next-mf2` plugin (0.1.1)	Early / unevaluated	—
Rust	none shipped as of 2026-04	—	This project fills the gap

A search of the ICU4X workspace for `messageformat`, `mf2`, `message_format` finds no existing code — this is a greenfield component.

2. Syntax (ABNF highlights)

Source files (message-format-wg repo): `spec/message.abnf`, `spec/syntax.md`.

Two top-level shapes

```
message           = simple-message / complex-message
simple-message    = o [simple-start pattern]
complex-message   = o *(declaration o) complex-body o
complex-body      = quoted-pattern / matcher
quoted-pattern    = "{{" pattern "}}"
```

- A **simple message** is plain text optionally interspersed with placeholders: `Hello, {$user}!` .
- A **complex message** begins with declarations and ends with either a single `{{…pattern…}}` or a `.match` matcher.

Declarations

```
declaration       = input-declaration / local-declaration
input-declaration = ".input"  o variable-expression
local-declaration = ".local" s variable o "=" o expression
```

- `.input {$amount :number minimumFractionDigits=2}` — annotate an external value (and optionally mark it as a selector).
- `.local $half = {$amount :math operand=half}` — derive a named local value from other expressions.
- Spec requires: every `.match` selector variable must have been declared with a function annotation (directly or transitively).

Expressions and placeholders

```
placeholder         = expression / markup
expression          = literal-expression / variable-expression / function-expression
literal-expression  = "{" o literal        [s function] *(s attribute) o "}"
variable-expression = "{" o variable       [s function] *(s attribute) o "}"
function-expression = "{" o function           *(s attribute) o "}"
function            = ":" identifier *(s option)
option              = identifier o "=" o (literal / variable)
attribute           = "@" identifier [o "=" o literal]
variable            = "$" name
literal             = quoted-literal / unquoted-literal
```

- Literals: unquoted (`name-char+`) or quoted (`|...|`) with `\\ \{ \} \|` escapes.
- Markup: `{#tag opts attrs}` , `{#tag /}` , `{/tag}` — three kinds `open` / `standalone` / `close` . Markup does not produce text; it produces a part.
- Attributes (`@key[=literal]`) are metadata only. They MUST NOT influence formatted output and MUST NOT be passed to function handlers.

Selection

```
matcher         = match-statement s variant *(o variant)
match-statement = ".match" 1*(s selector)
selector        = variable
variant         = key *(s key) o quoted-pattern
key             = literal / "*"
```

Constraints (enforced as *Data Model Errors*):

- ≥ 1 selector, ≥ 1 variant.
- Every variant's key count equals the selector count.

- Some variant must be all `*` (missing fallback variant ⇒ data-model error).
- No duplicate variant (identical key lists).
- Selector variables must resolve via a function declaration (missing selector annotation error).

Names, namespaces and bidi

- Names follow a Unicode identifier profile (TR31-based). Case sensitive.
- Identifiers can be namespaced: `ns:name`. The single-letter namespaces (`a:` … `z:`, `A:` … `Z:`) are reserved for future specification extensions — only `u:` is currently specified.
- Bidi isolate controls (LRI/RLI/FSI/PDI: `U+2066` … `U+2069`) and bidi marks (`U+061C`, `U+200E`, `U+200F`) are allowed as whitespace around names but must be stripped for comparison.

Escape sequences

Inside quoted literals: `\\ \{ \} \|`. Inside patterns (text): `\\ \{ \}`.
No other escapes — not even `\n`.

3. Data model

Source: `spec/data-model/README.md` (prose + TypeScript), `spec/data-model/message.json` (JSON Schema).
Summary in TypeScript notation (names match the interchange schema):

```ts
type Message = PatternMessage | SelectMessage;

interface PatternMessage {
  type: "message";
  declarations: Declaration[];
  pattern: Pattern;
}
interface SelectMessage {
  type: "select";
  declarations: Declaration[];
  selectors: VariableRef[];
  variants: Variant[];
}

type Declaration = InputDeclaration | LocalDeclaration;
interface InputDeclaration { type: "input";  name: string; value: VariableExpression; }
interface LocalDeclaration { type: "local";  name: string; value: Expression; }

type Pattern    = Array<string | Expression | Markup>;
type Expression = LiteralExpression | VariableExpression | FunctionExpression;

interface FunctionRef   { type: "function"; name: string; options: Map<string, Literal | VariableRef>; }
interface Markup        { type: "markup"; kind: "open" | "standalone" | "close"; name: string;
                          options: Map<string, Literal | VariableRef>; attributes: Attributes; }
type Attributes         = Map<string, Literal | true>;

interface CatchallKey   { type: "*"; value?: string; }
interface Variant       { keys: Array<Literal | CatchallKey>; value: Pattern; }
```

Notes:

- `VariableRef.name` and `InputDeclaration.name` do **not** include the leading `$`.

- Option *keys* must be unique within a single function application (data-model error otherwise). Option *values* of type `VariableRef` are resolved at runtime.
- Attribute values default to `true` when written without `=literal`.

4. Formatting algorithm

Source: `spec/formatting.md`.

Phases

1. **Parse** or validate the data model. Syntax errors and data-model errors are hard and happen before formatting begins.
2. **Build formatting context**: locale fallback chain, base directionality, input variable mapping, function handler registry, optional message-level fallback string.
3. **Resolve expressions** (lazy, call-by-need, evaluate each at most once):
   - Literal → character sequence after escape processing.
   - Variable → walk declarations to bind to either an `.input`, a `.local`, or an external input. Unbound ⇒ *Unresolved Variable* error.
   - Function expression → resolve operand, resolve option values, look up the handler in the registry, hand over `(context, operand, options) → resolved value`. Unknown function ⇒ *Unknown Function* error; handler-emitted errors ⇒ *Message Function Error* (Bad Operand / Bad Option / Unsupported Operation).
4. **Select pattern** (only for `SelectMessage`):
   - For each selector, resolve its annotated expression. Each selector contributes a typed resolved value that knows how to answer `match(key: Literal)` and `betterThan(keyA, keyB)`.
   - Build the matrix of variants, filter to those matching all selectors, pick the first in spec order after applying the per-selector `betterThan` ordering, fall back to the all-`*` variant otherwise.
   - A selector whose value does not support selection ⇒ *Bad Selector* error; only all-`*` variants may match.
5. **Format the selected pattern**:
   - Text → literal string (after escape processing, stripping no whitespace).
   - Expression → ask the resolved value for its string or "formatted parts".
   - Markup → emit structured part; no text output.
   - Apply bidi isolation where the context asks for it.

Fallback values

When an expression fails, the spec defines a fallback representation used in concatenated string output:

Form	Fallback string
`{	literal
`{unquoted :fn ...}`	`{unquoted}`
`{$name :fn ...}`	`{$name}`
`{:fn ...}`	`{:fn}`

Fallbacks preserve the original shape so translators and developers can see what failed, without crashing the render.

Bidi isolation

Default policy: when rendering a placeholder whose directionality differs from the surrounding pattern, wrap it with `LRI` / `RLI` / `FSI` and `PDI`. The implementation exposes this as an opt-out (`bidiIsolation = "none"`). See `spec/u-namespace.md` for `u:dir` .

`u:` namespace

- `u:id` — opaque identifier propagated into structured output parts. Ignored for string rendering; required for `formatToParts` equivalents.
- `u:dir` — `ltr | rtl | auto | inherit` . Controls bidi isolation. Error to use on markup.
- `u:locale` (Draft) — override locale for a single expression.

These options are stripped from the options map before the user-level function handler is called; the resolved value retains them.

5. Error taxonomy

Source: `spec/errors.md` . Four buckets; every error either aborts formatting (syntax / data-model) or emits during formatting and yields a fallback representation (resolution / message function):

Category	Emitted by	Examples	Output effect
Syntax Error	Parser	Unmatched `{{` , malformed `.match` , bad escape	Abort
Data Model Error	Validator	Missing Fallback Variant, Duplicate Declaration, Duplicate Option, Duplicate Variant, Variant Key Mismatch, Missing Selector Annotation	Abort
Resolution Error	Expression resolver	Unresolved Variable, Unknown Function, Bad Selector	Use fallback value
Message Function Error	Function handler	Bad Operand, Bad Option, Unsupported Operation	Use fallback value

All errors are reported — the spec is emphatic that implementations MUST record *every* error that occurred during a format, even when a fallback rendering was emitted. This shapes the public API: formatting returns both a rendered string AND the list of errors.

6. Required and recommended functions

Source: `spec/functions/{string,number,datetime}.md` + README.

Required (stable)

- `:string` — format and select. Selection uses NFC-normalised string compare against keys.
- `:number` — format and select numeric values.
- `:integer` — alias of `:number` with `maximumFractionDigits=0` behaviour plus integer-specific selection behavior.

Shared `:number` / `:integer` options: `select` (`plural` / `ordinal` / `exact`), `signDisplay` , `useGrouping` , `notation` , `compactDisplay` , `numberingSystem` , digit size (`minimumIntegerDigits` , min/max fraction, min/max significant),

rounding (`roundingPriority` , `roundingIncrement` , `roundingMode` , `trailingZeroDisplay`).

Recommended (draft)

- `:date` , `:time` , `:datetime` — ISO 8601 / Temporal PlainDateTime input; option skeletons (`dateStyle` , `timeStyle` , `fields` , `length` , `timeZone` , `calendar` , `hourCycle` , etc.).
- `:currency` / `:percent` / `:unit` / `:offset` — number-family decorators.
- `:math` — numerical transformation for selection (e.g. `operand=half`).

Implementations MUST support the stable set; the draft set is recommended.
Custom functions are allowed via the namespace mechanism.

7. The JSON conformance test suite

Location in the working-group repo: `test/tests/*.json`
(16 files at the time of cloning, plus `functions/*.json`).

Schema: `test/schemas/v0/tests.schema.json` — versioned `v0` .

Shape of a test case

```
{
  "$schema": "../schemas/v0/tests.schema.json",
  "scenario": "Pattern selection",
  "description": "Tests for pattern selection",
  "defaultTestProperties": { "locale": "und" },
  "tests": [
    {
      "src": ".input {$x :test:select} .match $x 1.0 {{1.0}} 1 {{1}} * {{other}}",
      "params": [{ "name": "x", "value": 1 }],
      "exp": "1"
    },
    {
      "src": ".local $x = {1 :test:select decimalPlaces=9} .match $x 1.0 {{1.0}} 1 {{1}} * {{bad-option-value}}",
      "exp": "bad-option-value",
      "expErrors": [{ "type": "bad-option" }, { "type": "bad-selector" }]
    }
  ]
}
```

Per-test fields worth noting:

- `src` — the message source.
- `params` — array of `{name, value}` input bindings.
- `exp` — expected formatted string (optional).
- `expParts` — expected formatToParts output with `type` , `value` , `kind` , `dir` , `locale` , `name` keys.
- `expErrors` — expected error types (by kebab-case tag).
- `locale` , `bidiIsolation` — override context properties.
- `only` , `srcs` , `tags` — harness conveniences.

A number of the fixtures (e.g. `pattern-selection.json`) rely on a `:test:select` / `:test:format` pair defined in the spec's appendix — implementations are expected to register these for conformance testing.

Fixture inventory

File	Coverage
`syntax.json`	Well-formed grammar accept set
`syntax-errors.json`	Grammar reject set
`data-model-errors.json`	Validation errors
`pattern-selection.json`	Matcher + best-match algorithm
`u-options.json`	`u:id`, `u:dir`, `u:locale`
`fallback.json`	Fallback substitution strings
`bidi.json`	Bidi isolates in source and output
`functions/string.json`	`:string` format + select
`functions/number.json`	`:number` format + select (plural/ordinal/exact)
`functions/integer.json`	`:integer`
`functions/date.json`	`:date` (draft)
`functions/time.json`	`:time` (draft)
`functions/datetime.json`	`:datetime` (draft)
`functions/currency.json`	`:currency` (draft)
`functions/percent.json`	`:percent` (draft)
`functions/offset.json`	`:offset` (draft)

8. Prior art: the `messageformat` npm package (v4+)

Source: https://www.npmjs.com/package/messageformat, GitHub monorepo.

Key observations that inform the Rust design:

- The JS library fully separated parsing, data-model, and runtime: `parseMessage(src) → Message`, `stringifyMessage(Message) → src`, `new MessageFormat(locales, src_or_Message, opts)`. ICU4X's design will follow the same three-stage split so users can ship a pre-parsed data model and skip the parser at runtime.
- Function registry is a plain map passed through options: `new MessageFormat('en', src, { functions: { 'ns:fn': handler } })`.
- Handler signature returns an object with both a formatter and a selector: `(locales, operand, options) → { value, selection? }`. The selector, when present, is a function that receives the set of variant keys and returns them ordered by preference — not a boolean match. This is the correct model and maps cleanly onto Rust trait methods (`format`, `select`).
- `format(values)` does *not* throw for resolution / function errors. It returns a string; callers pass an `onError` callback to be informed of the emitted error list. `formatToParts(values)` returns structured parts.
- Bidi isolation is on by default; `bidiIsolation: 'none'` turns it off. Implementation wraps every non-text part with isolates, respecting `u:dir`.

- The reference JS tests reuse the working-group JSON fixtures, proving the fixture format is consumable by third-party runners.
- Packages of interest in the monorepo:
  - `mf2/messageformat` — core formatter + data model tools.
  - `mf2/icu-mf1` — MF1→MF2 compiler (legacy migration).
  - `mf2/fluent` — Fluent (`.ftl`)→MF2 compiler.
  - `mf2/resources` — XLIFF 2 / MF2 resource tooling.

9. Fit with ICU4X

The ICU4X project is explicitly designed for client-side, resource-constrained environments: `no_std` , `#[non_exhaustive]` , optional `alloc` , zero-copy data via `zerovec` / `yoke` , compiled data baked in with `databake` . An MF2 implementation has a natural dependency graph:

- `icu_plurals` → `:number` plural-rule selection.
- `icu_decimal` + `fixed_decimal` → `:number` / `:integer` formatting.
- `icu_datetime` + `icu_time` → `:date` , `:time` , `:datetime` formatting.
- `icu_locale` / `icu_locale_core` → locale negotiation and `u:locale` .
- `icu_properties` / `icu_normalizer` → NFC compare for `:string` , bidi.
- `icu_pattern` → inspiration but **not** a drop-in: MF2's AST is richer than `SinglePlaceholderPattern` .
- `writeable::Writeable` → output trait: `format_to_write(..., &mut W)` .

An `icu_messageformat` crate therefore sits one rung above every other formatter crate and ties them together, which is why the component does not exist yet in 2.2.0: it would not have been buildable before the lower layers stabilised. They have now.

The boilerplate, lint, and feature-flag requirements (`no_std` , `alloc` , `serde` , `datagen` , `compiled_data` , `unstable`) match the template set out in `documents/process/boilerplate.md` and the `[workspace.lints]` table in the root `Cargo.toml` . The starting home is `components/experimental/src/messageformat/` (per the precedent of `relativetime` , `displaynames` , `personnames` etc.), graduating to a top-level `components/messageformat/` crate once the feature surface stabilizes.

10. Key open questions (carried into architecture)

1. **Return type of `format`** : `(String, Vec<FormatError>)` or `Writeable` + error sink? (Pick `Writeable` + sink for no_std.)
2. **Parsed representation**: own `Ast` vs. CST. The spec schema encourages AST; `messageformat` JS keeps a CST as an opt-in. Ship AST only for v1.
3. **Function registry mutation**: builder at construction, or add a "register" method? (Builder: `MessageFormatterBuilder::function(...)` .)
4. **Draft function gating**: `#[cfg(feature = "unstable")]` for `:date` / `:time` / `:datetime` until the spec promotes them.
5. **Fixture-driven tests**: point `md-tests` -style harness at the working-group repo, pinned to a commit, vendored into the icu4x tree for reproducible CI. See `messageformat-v2-implementation-details.md` .

Source files explicitly consulted:

`/Users/alexeylyakhov/projects/message-format-wg/spec/{intro.md,syntax.md,formatting.md,errors.md,appendices.md,message.abnf,u-namespac`

;
`spec/data-model/{README.md,message.json}`;
`spec/functions/{README.md,string.md,number.md,datetime.md}`;
`test/tests/{syntax,syntax-errors,data-model-errors,pattern-selection,u-options,fallback,bidi}.json`
and `test/tests/functions/{string,number,integer,date,time,datetime,currency,percent,offset}.json`;
`test/schemas/v0/tests.schema.json`;
ICU4X tree `Cargo.toml`, `components/experimental/{Cargo.toml,src/lib.rs}`,
`components/plurals/{src/lib.rs,src/provider.rs}`, `documents/process/boilerplate.md`.